

Aprendizaje Automático: Proyecto práctico

Creación de un chatbot con arquitecturas Seq2Seq y LSTMs

Manuel Pegalajar Cuéllar¹

¹Departamento de Ciencias de la Computación e Inteligencia Artificial. Universidad de Granada

Contenido

- 1 Introducción y Objetivos
- 2 Arquitectura Seq2Seq
- 3 Preprocesamiento de Datos
- 4 Implementación en Keras
- 5 Tareas del Proyecto



Contenido

- 1 Introducción y Objetivos
- 2 Arquitectura Seq2Seq
- 3 Preprocesamiento de Datos
- 4 Implementación en Keras
- 5 Tareas del Proyecto



¿Qué vamos a construir?

Un **Chatbot Generativo** capaz de mantener una conversación simple.

A diferencia de los chatbots basados en reglas o recuperación (retrieval-based), este modelo:

- Aprende a generar texto palabra por palabra (o caracter por caracter).
- No selecciona respuestas predefinidas de una base de datos.
- Utiliza Deep Learning para entender la secuencia de entrada y predecir la secuencia de salida.

Contenido

- 1 Introducción y Objetivos
- 2 Arquitectura Seq2Seq**
- 3 Preprocesamiento de Datos
- 4 Implementación en Keras
- 5 Tareas del Proyecto

Sequence-to-Sequence (Seq2Seq)

El problema se modela como una arquitectura **Encoder-Decoder**.

Encoder

Procesa la secuencia de entrada (pregunta) y la comprime en un **vector de contexto** (estado interno).

Decoder

Toma el vector de contexto y genera la secuencia de salida (respuesta) paso a paso.

El motor: LSTM (Long Short-Term Memory)

Para procesar secuencias de longitud variable y mantener “memoria” a largo plazo, usamos celdas **LSTM** (Redes Neuronales Recurrentes).

Por qué LSTM y no RNN simple:

- Evita el problema del desvanecimiento del gradiente.
- Capaz de aprender dependencias temporales largas (útil para frases complejas).
- Mantiene dos estados: *Hidden State* (h) y *Cell State* (c).

Contenido

- 1 Introducción y Objetivos
- 2 Arquitectura Seq2Seq
- 3 Preprocesamiento de Datos**
- 4 Implementación en Keras
- 5 Tareas del Proyecto



Preprocesamiento: Tokens Especiales

El Decoder necesita saber cuándo empezar a hablar y cuándo parar.

- `<start>`: Indica al decoder que inicie la generación.
- `<end>`: Indica que la frase ha terminado.

Transformación de Datos

Entrada (Encoder): "¿Cómo estás?"

Salida (Decoder Input): ‘‘<start> Estoy bien’’

Salida Esperada (Target): ‘‘Estoy bien <end>’’

Entrenamiento (Teacher Forcing)

- Se le pasa al Decoder la entrada correcta del paso anterior.
- Optimizamos para minimizar el error entre la palabra predicha y la real.

Inferencia (Chat)

- No tenemos la respuesta real.
- La predicción del paso t se convierte en la entrada del paso $t + 1$.
- El proceso es un bucle hasta encontrar `<end>`.

Contenido

- 1 Introducción y Objetivos
- 2 Arquitectura Seq2Seq
- 3 Preprocesamiento de Datos
- 4 Implementación en Keras**
- 5 Tareas del Proyecto



1. El Encoder (Entrenamiento)

Usaremos la API Funcional de Keras.

```
1  # Definir entradas
2  encoder_inputs = Input(shape=(None, num_encoder_tokens))
3
4  # Capa LSTM
5  encoder_lstm = LSTM(latent_dim, return_state=True)
6
7  # Descartamos la salida, nos quedamos con los estados (h, c)
8  encoder_outputs, state_h, state_c = encoder_lstm(encoder_inputs)
9  encoder_states = [state_h, state_c]
10
```

Nota: encoder_states es el “pensamiento” comprimido de la pregunta.

2. El Decoder (Entrenamiento)

El Decoder se inicializa con los estados del Encoder.

```
1     decoder_inputs = Input(shape=(None, num_decoder_tokens))
2
3     # LSTM configurada para devolver secuencias y estados
4     decoder_lstm = LSTM(latent_dim, return_sequences=True, return_state=
True)
5
6     # Importante: initial_state=encoder_states
7     decoder_outputs, _, _ = decoder_lstm(decoder_inputs,
8     initial_state=encoder_states)
9
10    decoder_dense = Dense(num_decoder_tokens, activation='softmax')
11    decoder_outputs = decoder_dense(decoder_outputs)
12
```

3. Modelo de Inferencia

Para chatear, necesitamos separar los modelos, ya que el bucle de generación es manual.

Modelo Encoder:

```
1 encoder_model = Model(encoder_inputs, encoder_states)
```

Modelo Decoder:

```
1 # Entradas de estado manuales para el bucle
2 decoder_state_input_h = Input(shape=(latent_dim,))
3 decoder_state_input_c = Input(shape=(latent_dim,))
4 decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]
5
6 # Reutilizamos las capas entrenadas (decoder_lstm, decoder_dense)
7 # ... logica de conexion ...
```

Contenido

- 1 Introducción y Objetivos
- 2 Arquitectura Seq2Seq
- 3 Preprocesamiento de Datos
- 4 Implementación en Keras
- 5 Tareas del Proyecto



Tareas a Realizar (Resumen)

- 1 **Preprocesamiento:** Tokenización, creación de diccionarios ($\text{char} \leftrightarrow \text{int}$) y One-Hot Encoding. No olvidar `<start>` y `<end>`.
- 2 **Entrenamiento:** Construir y entrenar el modelo 'keras.Model' completo.
- 3 **Inferencia:** Implementar la función `decode_sequence` que realice el bucle de generación.
- 4 **Ampliación:** Modificar el dataset (más frases, temática propia) o mejorar la interacción.
- 5 **Análisis:** Probar con frases fuera del entrenamiento y comentar limitaciones (vs Transformers/GPT).

- **Datos:** El dataset proporcionado es minúsculo. El modelo convergerá muy rápido (cuidado con el overfitting).
- **Shapes:** Vigila las dimensiones de los tensores (Batch, TimeSteps, Features).
- **Documentación:** Comenta cada bloque de código explicando *por qué* lo haces, no solo *qué* hace el código.

¡Mucho ánimo con el proyecto!

Aprendizaje Automático: Proyecto práctico

Creación de un chatbot con arquitecturas Seq2Seq y LSTMs

Manuel Pegalajar Cuéllar¹

¹Departamento de Ciencias de la Computación e Inteligencia Artificial. Universidad de Granada